

PostgreSQL 常见为什么

阿里云

digoal

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- https://github.com/digoal/blog/blob/master/201906/20190621_01.md
- 垃圾回收
- 或
- **FREEZE**
 - 系统中存在无休止的全数据范围更新的大表、大表未分区
 - 系统中存在无休止的全数据范围更新的大表、大表已分区（但是freeze年龄为错开设置，并且设置有足够多的autovacuum worker）
 - 未设置autovacuum delay SMOOTH参数

为什么

- 为什么有突发IO和CPU，但是与业务无关
- **为什么IO与CPU持续很高，但是与业务无关**
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- naptime 高频
- 回收不掉(standby feedback(且有慢事务, LONG SQL), 慢/dead Slot(catalog_xmin, 影响catalog垃圾回收), 长事务, 慢SQL, 慢2pc, 强制vacuum defer等)
- 无用功

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- **为什么表和索引会膨胀**
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- HEAP AM, MVCC机制?
- 垃圾没有及时回收
- autovacuum disabled
- threshold, scale too big
 - 表级、全局设置
- naptime too big
- autovacuum delay SMOOTH 参数过大
- autovacuum worker 太少
- autovacuum memory size 太小(存放不下所有reclaimed ctid), 导致重复扫描INDEX回收垃圾
- HOT(ctid change, same block chain)
- 大表未分区, 跟不上产生垃圾的速度
- 膨胀点
 - https://github.com/digoal/blog/blob/master/201907/20190701_01.md

1 全局catalog 膨胀点

哪些垃圾不能被回收？

- 1、年龄小于vacuum_defer_cleanup_age设置的垃圾记录
- 2、当前实例中最老事务快照之后产生的垃圾记录
- 3、SLOT catalog_xmin后产生的垃圾记录
- 4、备库开启了feedback后，备库中最老事务快照(包括catalog_xmin, global xmin)之后产生的垃圾记录

什么时候可能膨胀？

- 1、vacuum_defer_cleanup_age 设置太大
- 2、整个实例中的：长事务, 慢SQL, 慢2pc,
- 3、慢/dead slot(catalog_xmin, 影响catalog垃圾回收),
- 4、standby 开启了 feedback (且standby有慢事务, LONG SQL, 慢/dead slot),

2 库级catalog 膨胀点

哪些垃圾不能被回收？

- 1、年龄小于vacuum_defer_cleanup_age设置的垃圾记录
- 2、当前数据库中最老事务快照之后产生的垃圾记录
- 3、备库开启了feedback后，备库返回的最老事务快照(包括catalog_xmin, global xmin)之后产生的垃圾记录
- 4、SLOT catalog_xmin后产生的垃圾记录(create table, drop table, pg_class, pg_att等)。影响全局（所有DB）

什么时候可能膨胀？

- 1、vacuum_defer_cleanup_age 设置太大
- 2、当前数据库中的：长事务, 慢SQL, 慢2pc,
- 3、standby 开启了 feedback (且standby有慢事务, LONG SQL, 慢/dead slot),
- 4、慢/dead slot(catalog_xmin, 影响catalog垃圾回收),

普通对象 膨胀点

用户创建的表、物化视图、索引等。

哪些垃圾不能被回收？

- 1、年龄小于vacuum_defer_cleanup_age设置的垃圾记录
- 2、当前数据库中最老事务快照之后产生的垃圾记录
- 3、备库开启了feedback后，备库返回的最老事务快照(仅指 global xmin)之后产生的垃圾记录。(catalog xmin无影响)

什么时候可能膨胀？

- 1、vacuum_defer_cleanup_age 设置太大
- 2、当前数据库中的：长事务, 慢SQL, 慢2pc,
- 3、standby 开启了 feedback (且standby有慢事务, LONG SQL),

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- **为什么STANDBY有延迟(send,replay)**
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- 通常延迟1MS内
 - https://github.com/digoal/blog/blob/master/201803/20180326_01.md
- primary, standby网络带宽
- standby端IO能力
- standby如何回放? (wal buffer+block, wal buffer+shared buffer)

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- **为什么会雪崩**
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- lock
- 大锁被已有长事务小锁堵塞
- 高并发小锁等大锁
- 避免雪崩
 - lock, statement timeout

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- **为什么要垃圾回收**
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- mvcc 了解
- hot 了解

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- **为什么要freeze**
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- xid 32bit 循环使用
- frozen xid
- normal xid

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- **为什么有高水位**
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- rowid->citd
- index item (think how to point to tuple?)

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- **为什么push\pull大量数据的读写慢**
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- 高并发、simple query
- prepare
- group commit
- async commit
- copy
- batch
- begin commit

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- **有几种索引、该如何选择索引**
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

索引接口的选择

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ [ IF NOT EXISTS ] name ] ON [ ONLY ] table_name [ USING method ]  
    ( { column_name | ( expression ) } [ COLLATE collation ] [ opclass ] [ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ...] )  
    [ INCLUDE ( column_name [, ...] ) ]  
    [ WITH ( storage_parameter = value [, ...] ) ]  
    [ TABLESPACE tablespace_name ]  
    [ WHERE predicate ]
```

select * from pg_am;

<https://www.postgresql.org/docs/devel/static/sql-createindex.html>

https://github.com/digoal/blog/blob/master/201706/20170627_01.md

自动选择索引接口:

https://github.com/digoal/blog/blob/master/201706/20170617_01.md

索引接口的选择

- BTree
 - =, >, >=, <, <=、排序
- Hash (KEY值为HASH值, 可以适合很长的字符串)
 - =
- GIN
 - 多值类型(数组、全文检索、枚举、网络地址类型): 包含、相交
 - JSON类型: 包含 <https://www.postgresql.org/docs/devel/static/datatype-json.html#JSON-INDEXING>
 - 普通类型(通过btree_gin 插件支持): 与B-Tree类似
 - 字符串(通过pg_trgm 插件支持): 模糊查询、相似查询
 - 多列: 任意列组合查询
- GiST
 - 空间类型: 方位(上、下、左、右), 空间关系(相交、包含), 空间距离排序(KNN)
 - 范围数据: =, &&, <@, @>, <<, >>, -|- , &<, and &>
 - 普通类型(通过btree_gist 插件支持): 与B-Tree类似, 增加空间类型类似操作符
 - 数组类型(通过intarray 插件支持): 与GIN类似
 - 多列: 任意列组合查询
- SPGiST
 - 平面几何类型: 与GiST类似
 - 范围类型: 与GiST类似

索引接口的选择

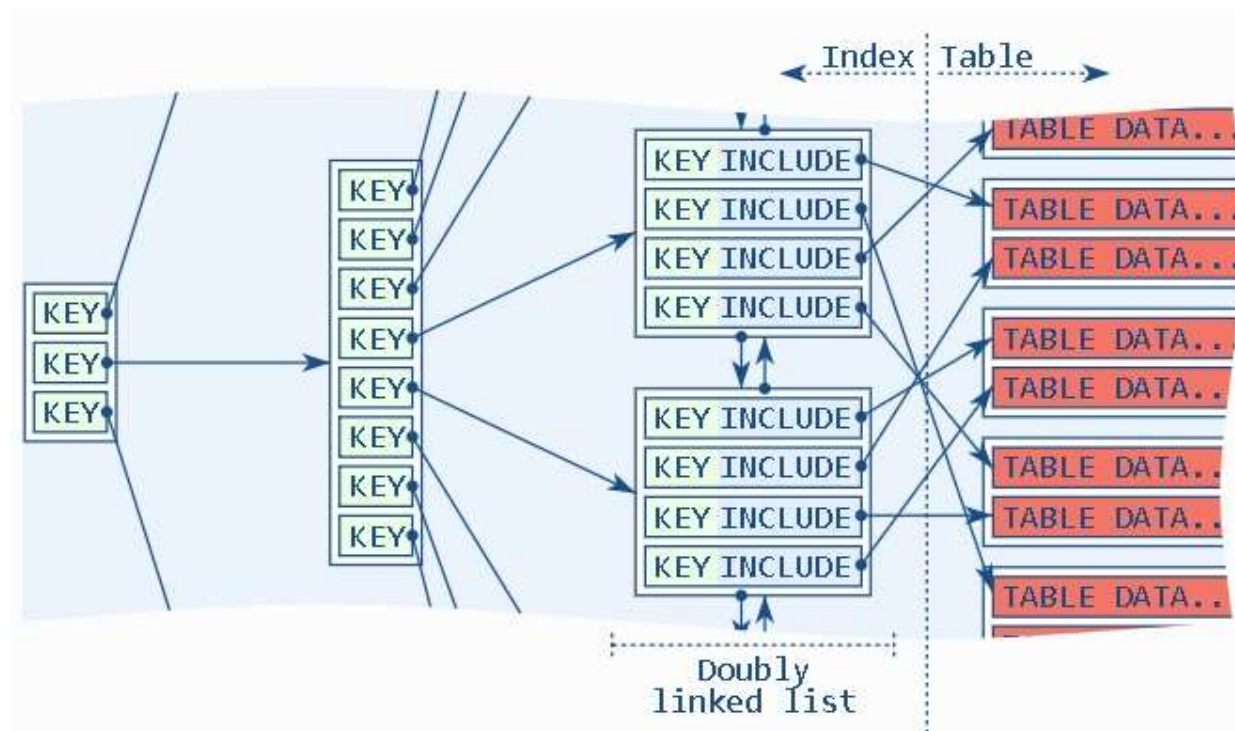
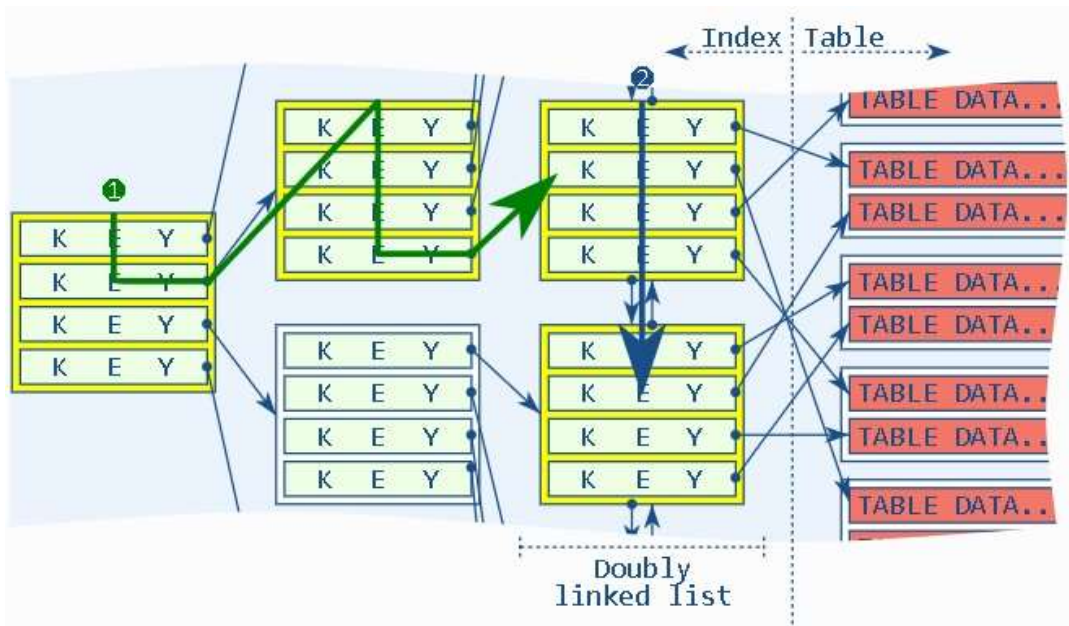
- RUM
 - 多值类型(数组、全文检索类型): 包含、相交、相似排序
 - 普通类型: 与B-Tree类似
- BRIN
 - 适合 线性数据、时序数据、（HEAP PAGE之间边界清晰的数据）。
 - 普通类型: 与B-Tree类似
 - 空间类型: 包含
- Bloom
 - 多列: 任意列组合, 等值查询
- 表达式索引
 - 搜索条件为表达式时。
 - `where st_makepoint(x,y) op ?`
 - `create index idx on tbl USING gist ( (st_makepoint(x,y)) );`
- 条件索引(定向索引)
 - 搜索时, 强制过滤某些条件时。
 - `where status='active' and col=?` 。
 - `create index idx on tbl (col) where status='active';` -- 激活用户占比70% , 只针对这些数据进行索引
 - 监控系统例子 `select x from tbl where temp>60;` -- 99, 1% 异常数据

索引接口的选择

- include索引

btree index include索引叶子附加属性

- https://github.com/digoal/blog/blob/master/201905/20190503_03.md
- 类似B+tree数据聚集,
- create index idx_t1_1 on t1 (id) include(c1,c2,c3,info,crt_time);



为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- **如何在线降低水位**
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- vacuum full (access exclusive lock)
- pg_repack
- pg_reorg

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- **如何在线加索引**
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- reindex currently

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- auto_explain
- pg wait sample
 - https://github.com/digoal/blog/blob/master/201901/20190125_02.md
- pg_stat_activity(当前状态)
- index advisor



为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈

- **如何找出捣蛋鬼**
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

TOP SQL

pg_stat_statements
插件

View "public.pg_stat_statements"		
Column	Type	Collation
userid	oid	
dbid	oid	
queryid	bigint	
query	text	
calls	bigint	
total_time	double precision	
min_time	double precision	
max_time	double precision	
mean_time	double precision	
stddev_time	double precision	
rows	bigint	
shared_blks_hit	bigint	
shared_blks_read	bigint	
shared_blks_dirtied	bigint	
shared_blks_written	bigint	
local_blks_hit	bigint	
local_blks_read	bigint	
local_blks_dirtied	bigint	
local_blks_written	bigint	
temp_blks_read	bigint	
temp_blks_written	bigint	
blk_read_time	double precision	
blk_write_time	double precision	

TOP SQL

- https://github.com/digoal/blog/blob/master/201704/20170424_06.md
- create extension pg_stat_statements;
- 最耗时 SQL (整体数据库优化, 建议以此结果逐条优化, 效果立竿见影)
- 总最耗时 SQL TOP 5
select dbid, query from pg_stat_statements order by total_time desc limit 5;
- 其他
 - 最耗IO SQL
 - 总最耗IO SQL TOP 5
select dbid, query from pg_stat_statements order by (blk_read_time+blk_write_time) desc limit 5;
 - 响应时间抖动最严重 SQL
select dbid, query from pg_stat_statements order by stddev_time desc limit 5;
 - 最耗共享内存 SQL
select dbid, query from pg_stat_statements order by (shared_blks_hit+shared_blks_dirtied) desc limit 5;
 - 最耗临时空间 SQL
select dbid, query from pg_stat_statements order by temp_blks_written desc limit 5;

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

数据分布离散，使用BTREE索引顺序返回导致的IO访问放大

- drop table test;
- create unlogged table test (gid int, c1 int, c2 int);
- insert into test select random()*10000 , 1, 1 from generate_series(1,10000000);
- create index idx_test_1 on test (gid);
- vacuum analyze test;
- vi test.sql
- \set id random(1,10000)
- select * from test where gid=:id;
- 一次范围1000条记录，数据离散，所以需要扫描1000个数据块。
- pgbench -M prepared -n -r -P 1 -f ./test.sql -c 32 -j 32 -T 120

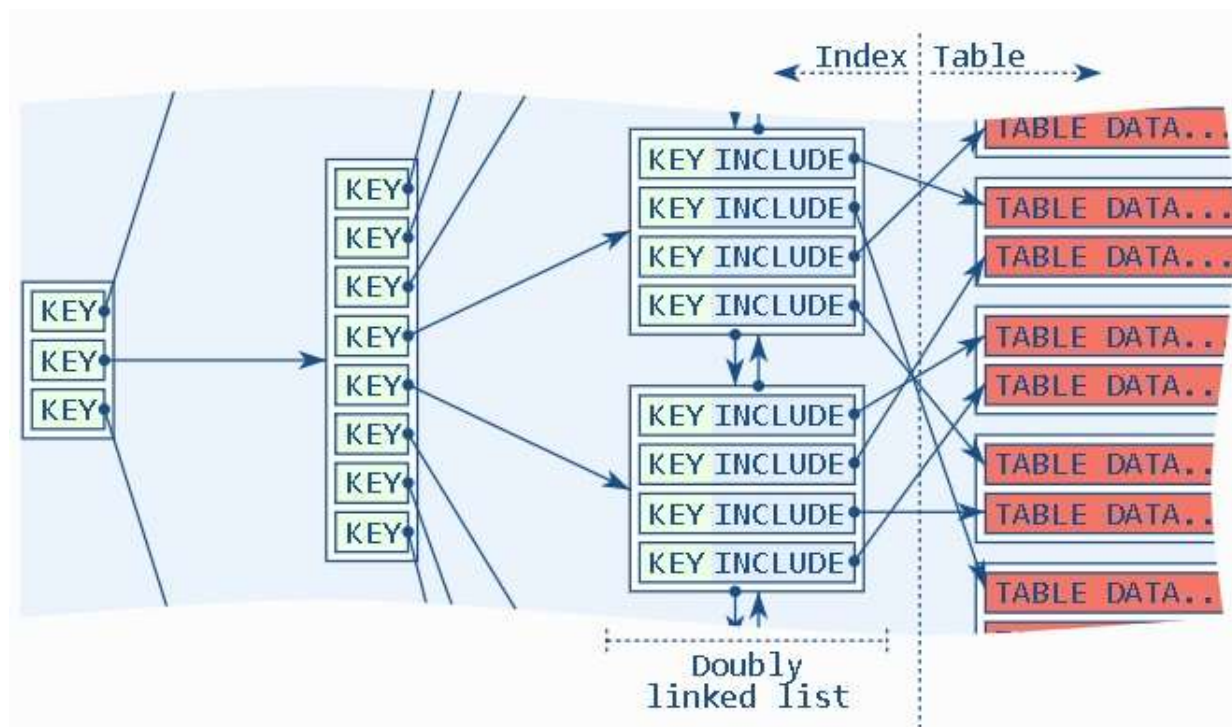
```
progress: 2.0 s, 19992.5 tps, lat 1.601 ms stddev 0.142
progress: 3.0 s, 20136.6 tps, lat 1.589 ms stddev 0.137
progress: 4.0 s, 20158.0 tps, lat 1.588 ms stddev 0.137
progress: 5.0 s, 20057.0 tps, lat 1.595 ms stddev 0.344
progress: 6.0 s, 20236.7 tps, lat 1.581 ms stddev 0.136
progress: 7.0 s, 20220.3 tps, lat 1.583 ms stddev 0.144
```


优化手段1: 聚集存储

```
postgres=# cluster test using idx_test_1 ;  
CLUSTER
```

```
progress: 2.0 s, 42686.0 tps, lat 0.750 ms stddev 0.054  
progress: 3.0 s, 42952.0 tps, lat 0.745 ms stddev 0.033  
progress: 4.0 s, 43179.0 tps, lat 0.741 ms stddev 0.046  
progress: 5.0 s, 43328.1 tps, lat 0.739 ms stddev 0.036  
progress: 6.0 s, 43452.7 tps, lat 0.736 ms stddev 0.037  
progress: 7.0 s, 43246.8 tps, lat 0.740 ms stddev 0.169  
progress: 8.0 s, 43630.2 tps, lat 0.734 ms stddev 0.035  
progress: 9.0 s, 43320.4 tps, lat 0.738 ms stddev 0.256
```

- 优化手段二，记录聚合
 - https://github.com/digoal/blog/blob/master/201812/20181209_01.md
 - https://github.com/digoal/blog/blob/master/201812/20181207_01.md
- 优化手段三，including index
 - https://github.com/digoal/blog/blob/master/201905/20190503_03.md



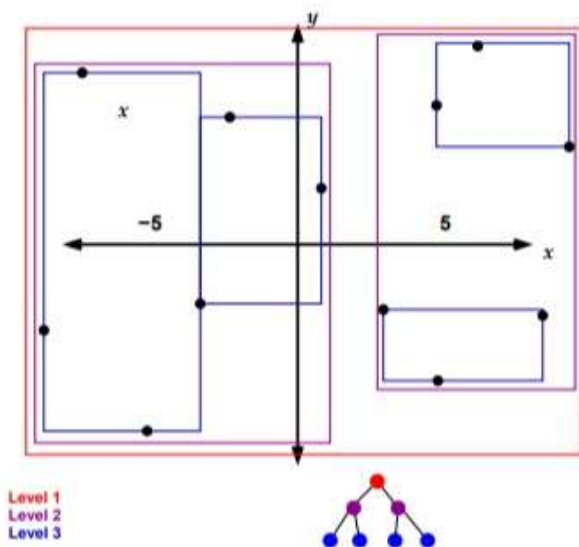
为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- **为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询**
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

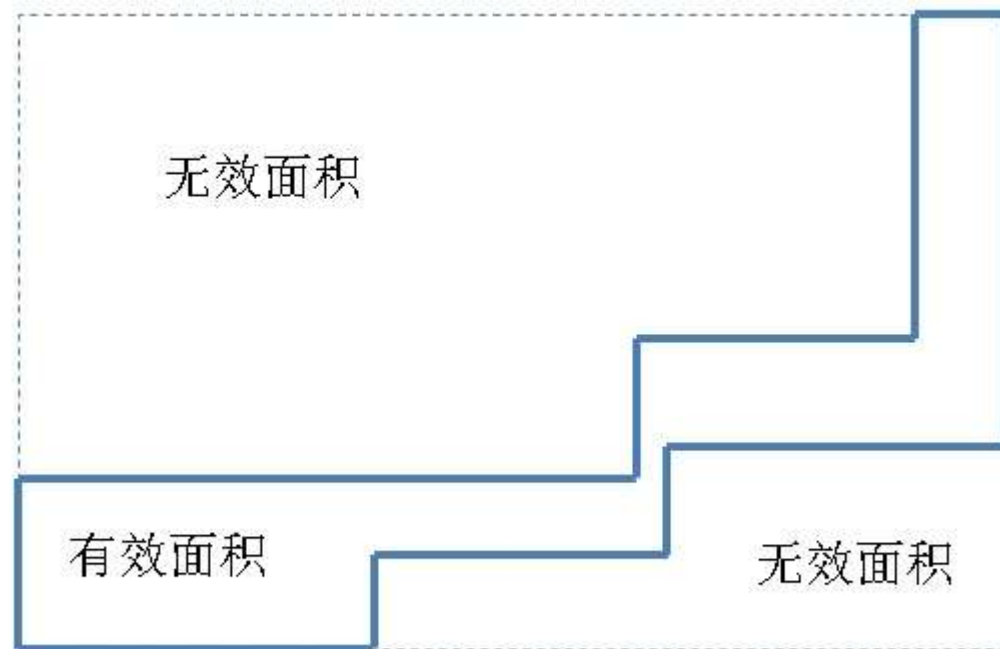
RTREE数据分布导致的IO放大和FILTER

- https://github.com/digoal/blog/blob/master/201710/20171005_01.md
- https://github.com/digoal/blog/blob/master/201710/20171004_01.md

R-Tree Indexes Bounding Boxes



Geographic objects (lines, polygons) also can appear in r-tree indexes. based on their own bounding boxes.



空间搜索的两个可以优化的点，原理如下：

- 1、空间数据在存储时乱序存放，导致搜索一批数据时扫描的数据块很多。（点查感觉不到这个问题。）
- 2、PostGIS的GiST空间索引，采用了BOUND BOX作为KEY，搜索时也是使用对象的BOUND BOX进行搜索，因此当对象是长条条时，可能造成大量的BOUND BOX空洞，放大了扫描范围（对st_contains, st_within来说），增加了CPU过滤的开销。

优化手段1：空间聚集，解决IO放大问题。

优化手段2：对输入条件（长条条的多边形）进行SPLIT，降低BOUND BOX放大引入的扫描范围（对st_contains, st_within来说）放大的问题。

数据量：1000万。

点面判断（长条形多边形，或者离散多个多边形对象覆盖的空间对象）。

对比	优化前	优化1（空间聚集）	优化1,2(SPLIT多边形)
访问数据块数量	35323	1648	243
被过滤的无效记录条数	26590	26590	0

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- **为什么OFFSET大会慢**
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

OFFSET大的问题

- drop table test;
- create unlogged table test (gid int, c1 int, c2 int, crt_time timestamp);
- insert into test select random()*100 , 1, 1 , clock_timestamp() from generate_series(1,10000000);
- create index idx_test_1 on test (gid,crt_time);
- vacuum analyze test;
- explain (analyze,verbose,timing, costs,buffers,summary) select * from test where gid=2 order by crt_time offset 80000 limit 1;

```
postgres=# explain (analyze,verbose,timing,costs,buffers,summary) select * from test where gid=2 order by crt_time offset 80000 limit 1;  
QUERY PLAN
```

```
-----  
Limit  (cost=51254.91..51255.55 rows=1 width=20) (actual time=61.244..61.246 rows=1 loops=1)
```

```
Output: gid, c1, c2, crt_time
```

```
Buffers: shared hit=40756
```

```
->  Index Scan using idx_test_1 on public.test  (cost=0.43..63428.49 rows=99001 width=20) (actual time=0.040..57.360 rows=80001 loops=1)
```

```
Output: gid, c1, c2, crt_time
```

```
Index Cond: (test.gid = 2)
```

```
Buffers: shared hit=40756
```

```
Planning Time: 0.094 ms
```

```
Execution Time: 61.268 ms
```

```
(9 rows)
```


- https://github.com/digoal/blog/blob/master/201605/20160506_01.md
- key
- 游标

原始结构与SQL：

```
create table test(id int, c1 int, c2 int, c3 int, c4 int, crt_time timestamp);

create index idx_test_1 on test(c1, crt_time);

insert into test select id, random()*10, random()*10, random()*10, random()*10, clock_timestamp() from generate_series(1, 100000);

select * from test where c1=1 and c2 between 1 and 10 order by crt_time limit 10 offset ?;
```

增加PK，SQL改成如下：

```
create table test1(pk serial8 primary key, -- 新增PK
id int, c1 int, c2 int, c3 int, c4 int, crt_time timestamp);

create index idx_test1_1 on test1(c1, crt_time, pk); -- 索引修改

insert into test1 (id,c1,c2,c3,c4,crt_time) select id, random()*10, random()*10, random()*10, random()*10, clock_timestamp() from generate_series(1, 100000);

select * from test1 where c1=1 and c2 between 1 and 10
and crt_time>=? -- 位点为上一次请求的最大crt_time
and pk>? -- 位点为上一次请求的最大pk值
order by crt_time,pk limit 10; -- 改成order by crt_time,pk，同时去掉offset
```

offset 100000后,性能对比:

```
explain (analyze,verbose,timing, costs,buffers)
select * from test
where c1=1 and c2 between 1 and 10 order by crt_time limit 10 offset 100000;
```

QUERY PLAN

Limit (cost=11739.53..11740.70 rows=10 width=28) (actual time=53.373..53.379 rows=10 loops=1)

Output: id, c1, c2, c3, c4, crt_time

Buffers: shared hit=8506

-> Index Scan using idx_test_1 on public.test (cost=0.56..110585.65 rows=942034 width=28) (actual time=0.039..45.039 rows=10 loops=1)

Output: id, c1, c2, c3, c4, crt_time

Index Cond: (test.c1 = 1)

Filter: ((test.c2 >= 1) AND (test.c2 <= 10))

Rows Removed by Filter: 5222

Buffers: shared hit=8506

Planning time: 0.135 ms

Execution time: 53.406 ms

(11 rows)

对比

```
explain (analyze,verbose,timing, costs, buffers)
select * from test1
where c1=1 and c2 between 1 and 10
    and crt_time>='2018-07-25 18:31:14.860328' -- 位点为上一次请求的最大crt_time
    and pk>1048412 -- 位点为上一次请求的最大pk值
order by crt_time,pk limit 10; -- 改成order by crt_time,pk, 同时去掉offset
```

QUERY PLAN

```
-----
Limit (cost=0.56..2.17 rows=10 width=36) (actual time=0.042..0.049 rows=10 loops=1)
  Output: pk, id, c1, c2, c3, c4, crt_time
  Buffers: shared hit=9
  -> Index Scan using idx_test1_1 on public.test1 (cost=0.56..124129.55 rows=770820 width=36) (actual time=0.040..0.048 rows=10 loops=1)
    Output: pk, id, c1, c2, c3, c4, crt_time
    Index Cond: ((test1.c1 = 1) AND (test1.crt_time >= '2018-07-25 18:31:14.860328'::timestamp without time zone))
    Filter: ((test1.c2 >= 1) AND (test1.c2 <= 10))
    Buffers: shared hit=9
Planning time: 0.174 ms
Execution time: 0.075 ms
(10 rows)
```

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- **为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险**
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- 膨胀点危害
- https://github.com/digoal/blog/blob/master/201907/20190701_01.md
- bloat
- xid wrapper problem
- CPU、IO无用功
- 实践
 - 监控
 - snapshot too old
 - statement idle in transaction timeout
 - zheap\zedstore

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- **为什么REDO日志没有清理**
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- archive failed
- user defined archive BUG
- wal keep segments
- max wal size
- slot slow(dead) , restart_lsn, (physical | logical replication)
- checkpoint target too big

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- **为什么GIN有时不快**
- 为什么BRIN有时不快
- 为什么执行计划不正确

gin pending, 倒排查询慢

- create or replace function gen_randarr(int,int) returns int[] as \$\$
- select array(select (random()*\$1)::int from generate_series(1,\$2));
- \$\$ language sql strict;
- drop table test;
- create unlogged table test (id int, c1 int[]);
- create index idx_test_1 on test using gin (c1);
- vi test.sql
- \set id random(1,1000000000)
- insert into test values (:id, gen_randarr(1000,10));
- pgbench -M prepared -n -r -P 1 -f ./test.sql -c 32 -j 32 -T 120

gin pending, 倒排查询慢

- create extension pageinspect;
- <https://www.postgresql.org/docs/11/pageinspect.html>
- SELECT * FROM gin_metapage_info(get_raw_page('idx_test_1', 0));

```
postgres=# SELECT * FROM gin_metapage_info(get_raw_page('idx_test_1', 0));
-[ RECORD 1 ]-----+-----
pending_head      | 67070
pending_tail      | 67412
tail_free_size    | 6180
n_pending_pages   | 823
n_pending_tuples  | 13262
n_total_pages     | 2
n_entry_pages     | 1
n_data_pages      | 0
n_entries         | 0
version           | 2
```

有较大pending list时，由于查询时需要遍历pendinglist进行合并，所以查询RT会比较高

```
postgres=# explain (analyze,verbose,timing, costs,buffers,summary) select count(*) from test where c1 @> array[1,2];
               QUERY PLAN
-----
Aggregate  (cost=2261.52..2261.53 rows=1 width=8) (actual time=22.340..22.341 rows=1 loops=1)
  Output: count(*)
  Buffers: shared hit=1934
    -> Bitmap Heap Scan on public.test  (cost=27.82..2256.45 rows=2028 width=0) (actual time=19.114..22.194 rows=1709 loops=1)
        Recheck Cond: (test.c1 @> '{1,2}'::integer[])
        Heap Blocks: exact=1699
        Buffers: shared hit=1934
        -> Bitmap Index Scan on idx_test_1  (cost=0.00..27.31 rows=2028 width=0) (actual time=18.915..18.915 rows=1709 loops=1)
            Index Cond: (test.c1 @> '{1,2}'::integer[])
            Buffers: shared hit=235
  Planning Time: 0.111 ms
  Execution Time: 22.375 ms
(12 rows)
```

```
postgres=# explain (analyze,verbose,timing, costs,buffers,summary) select count(*) from test where c1 @> array[1];
               QUERY PLAN
-----
Aggregate  (cost=378779.78..378779.79 rows=1 width=8) (actual time=3422.348..3422.348 rows=1 loops=1)
  Output: count(*)
  Buffers: shared hit=131025
    -> Bitmap Heap Scan on public.test  (cost=1771.64..378238.45 rows=216534 width=0) (actual time=32.767..3406.933 rows=189943 loops=1)
        Recheck Cond: (test.c1 @> '{1}'::integer[])
        Rows Removed by Index Recheck: 9193163
        Heap Blocks: exact=15374 lossy=115564
        Buffers: shared hit=131025
        -> Bitmap Index Scan on idx_test_1  (cost=0.00..1717.50 rows=216534 width=0) (actual time=30.111..30.111 rows=189943 loops=1)
            Index Cond: (test.c1 @> '{1}'::integer[])
            Buffers: shared hit=80
  Planning Time: 0.841 ms
  Execution Time: 3422.904 ms
(13 rows)
```

postgres=# vacuum analyze test;

vacuum table后，查询索引pending lists，解决。

```
postgres=# SELECT * FROM gin_metapage_info(get_raw_page('idx_test_1', 0));
-[ RECORD 1 ]-----+-----
pending_head      | 4294967295
pending_tail      | 4294967295
tail_free_size    | 0
n_pending_pages   | 0
n_pending_tuples  | 0
n_total_pages     | 78429
n_entry_pages     | 456
n_data_pages      | 77051
n_entries         | 1001
version           | 2
```

```
postgres=# vacuum analyze test;
```

```
VACUUM
```

```
postgres=# explain (analyze,verbose,timing, costs, buffers, summary) select count(*) from test where c1 @> array[1,2];
```

```
QUERY PLAN
```

```
-----  
Aggregate  (cost=2082.57..2082.58 rows=1 width=8) (actual time=19.734..19.734 rows=1 loops=1)
```

```
Output: count(*)
```

```
Buffers: shared hit=243
```

```
-> Bitmap Heap Scan on public.test  (cost=189.31..2078.28 rows=1717 width=0) (actual time=19.326..19.607 rows=1709 loops=1)
```

```
Recheck Cond: (test.c1 @> '{1,2}'::integer[])
```

```
Heap Blocks: exact=1699
```

```
Buffers: shared hit=243
```

```
-> Bitmap Index Scan on idx_test_1  (cost=0.00..188.88 rows=1717 width=0) (actual time=19.100..19.100 rows=1709 loops=1)
```

```
Index Cond: (test.c1 @> '{1,2}'::integer[])
```

```
Buffers: shared hit=235
```

```
Planning Time: 0.201 ms
```

```
Execution Time: 19.769 ms
```

```
(12 rows)
```

```
postgres=# explain (analyze,verbose,timing, costs, buffers, summary) select count(*) from test where c1 @> array[1];
```

```
QUERY PLAN
```

```
-----  
Aggregate  (cost=369064.84..369064.85 rows=1 width=8) (actual time=2413.556..2413.557 rows=1 loops=1)
```

```
Output: count(*)
```

```
Buffers: shared hit=115647
```

```
-> Bitmap Heap Scan on public.test  (cost=1623.61..368569.48 rows=198144 width=0) (actual time=33.011..2398.791 rows=189943 loops=1)
```

```
Recheck Cond: (test.c1 @> '{1}'::integer[])
```

```
Rows Removed by Index Recheck: 9193163
```

```
Heap Blocks: exact=15374 lossy=115564
```

```
Buffers: shared hit=115647
```

```
-> Bitmap Index Scan on idx_test_1  (cost=0.00..1574.08 rows=198144 width=0) (actual time=30.409..30.409 rows=189943 loops=1)
```

```
Index Cond: (test.c1 @> '{1}'::integer[])
```

```
Buffers: shared hit=80
```

```
Planning Time: 0.154 ms
```

```
Execution Time: 2413.598 ms
```

```
(13 rows)
```

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net
还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，
为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，
standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么**BRIN**有时不快
- 为什么执行计划不正确

- overlap data between many pages

为什么

- 为什么有突发IO和CPU，但是与业务无关
- 为什么IO与CPU持续很高，但是与业务无关
- 为什么表和索引会膨胀
- 为什么STANDBY有延迟(send,replay)
- 为什么会雪崩
- 为什么要垃圾回收
- 为什么要freeze
- 为什么有高水位
- 为什么push\pull大量数据的读写慢
- 有几种索引、该如何选择索引
- 如何在线降低水位
- 如何在线加索引
- 如何发现过去、现在、未来的瓶颈
- 如何找出捣蛋鬼
- 为什么时序类搜索可能有IO放大，除了cpu,io,net还有哪些隐藏的瓶颈，为什么要聚集
- 为什么空间搜索有IO放大，为什么要数据聚集，为什么要分裂查询
- 为什么OFFSET大会慢
- 为什么慢SQL，空闲事务，长事务，2PC，慢SLOT，standby feedback，强制vacuum age defer有风险
- 为什么REDO日志没有清理
- 为什么GIN有时不快
- 为什么BRIN有时不快
- 为什么执行计划不正确

- 统计信息维度多少
- 多列条件混合统计信息维度多少
- 统计信息实时性
- 优化器公式环境修正参数
- 多表JOIN自动修正顺序相关参数
- 多表JOIN遗传算法

优化器因子系数参数不对导致使用 bitmapscan, IO放大, CPU放大

- drop table test;
- create unlogged table test (id int, info text, crt_time timestamp);
- insert into test select random()*1000, md5(random()::text), clock_timestamp() from generate_series(1,10000000);
- create index idx_test_1 on test (id);
- cluster test USING idx_test_1;
- set random_page_cost=4; -- 随机扫描1 PAGE的成本
- vacuum analyze test;
- explain (analyze,verbose,timing,costs,buffers,summary) select count(*) from test where id<20;

```
postgres=# explain (analyze,verbose,timing,stats,buffers,summary) select count(*) from test where id<20;  
               QUERY PLAN
```

```
-----  
Aggregate  (cost=212756.58..212756.59 rows=1 width=8) (actual time=44.710..44.710 rows=1 loops=1)  
  Output: count(*)  
  Buffers: shared hit=2352  
  -> Bitmap Heap Scan on public.test  (cost=3179.20..212332.46 rows=169647 width=0) (actual time=8.626..31.914 rows=194479 loops=1)  
        Recheck Cond: (test.id < 20)  
        Heap Blocks: exact=1818  
        Buffers: shared hit=2352  
        -> Bitmap Index Scan on idx_test_1  (cost=0.00..3136.79 rows=169647 width=0) (actual time=8.401..8.401 rows=194479 loops=1)  
              Index Cond: (test.id < 20)  
              Buffers: shared hit=534
```

```
Planning Time: 0.089 ms
```

```
Execution Time: 44.746 ms
```

```
(12 rows)
```

调低随机扫描1 PAGE成本后，使用了索引扫描。没有recheck.
recheck消耗大量CPU，如果并发高，容易成为CPU瓶颈。

```
postgres=# set random_page_cost=1.1;  
SET
```

```
postgres=# explain (analyze,verbose,timing, costs,buffers,summary) select count(*) from test where id<20;  
                                QUERY PLAN  
-----  
Aggregate  (cost=101726.22..101726.23 rows=1 width=8) (actual time=41.677..41.677 rows=1 loops=1)  
  Output: count(*)  
  Buffers: shared hit=2352  
    -> Index Scan using idx_test_1 on public.test  (cost=0.43..101302.10 rows=169647 width=0) (actual time=0.014..28.787 rows=194479 loops=1)  
        Output: id, info, crt_time  
        Index Cond: (test.id < 20)  
        Buffers: shared hit=2352  
Planning Time: 0.036 ms  
Execution Time: 41.694 ms  
(9 rows)
```


为什么系统表会膨胀

- PG暂时没有全局临时表
- 每次使用需要新建（会话中除外，新会话需要）
- 每个临时表都会对应元数据(pg_class,...)
- 系统退出后删除，导致垃圾

pluggable AM

- zheap
 - <https://github.com/EnterpriseDB/zheap>
 - <https://github.com/EnterpriseDB/zheap/tree/master/src/backend/access/zheap>
- 64-bit transaction ids
- zheap will prevent bloat
 - (a) by allowing in-place updates in common cases and
 - (b) by reusing space as soon as a transaction that has performed a delete or non-in-place-update has committed.
- Reduce write amplification both by avoiding rewrites of heap pages and by making it possible to do an update that touches indexed columns without updating every index.
- Reduce the tuple size by
 - (a) shrinking the tuple header and
 - (b) eliminating most alignment padding.
- In-place updates will be supported except when
 - (a) the new tuple is larger than the old tuple and the increase in size makes it impossible to fit the larger tuple onto the same page or
 - (b) some column is modified which is covered by an index that has not been modified to support “delete-marking”.
 - We have not begun work on delete-marking support for indexes yet, but intend to support it at least for btree indexes.

pluggable AM

- zedstore
- <https://github.com/greenplum-db/postgres/tree/zedstore/src/backend/access/zedstore>
 - * Performance improvement for queries selecting subset of columns (reduced IO).
 - * Reduced on-disk footprint compared to heap table. Shorter tuple headers and also leveraging compression of similar type data
 - * Be first-class citizen in the Postgres architecture (tables data can just independently live in columnar storage) and not be at arm's length through an opaque interface.
 - * Fully MVCC compliant - basically all operations supported similar to heap, like update, delete, serializable transactions etc...
 - * All Indexes supported
 - * Hybrid row-column store, where some columns are stored together, and others separately. Provide flexibility of granularity on how to divide the columns. Columns accessed together can be stored together.
 - * Provide better control over bloat (using zheap)
 - * Eliminate need for separate toast tables
 - * Faster add / drop column or changing data type of column by avoiding full rewrite of the table.